

Conversational EDA Chatbot — and — E-Commerce Customer Churn Prediction

A Production-Grade Data Science Project

Tuyiramye Christian (2517025) • Dushime Pacifique (2517004)

Supervisor: Prof. Dr. Jebran Khan

Kyungdong University Global • Department of AI • Spring 2025

- 1. Motivation & Problem Statement
- 2. Objectives
- 3. System Architecture
- 4. Phase A — SQL Agent
- 5. Phase B — Visual Router & Pandas Agent
- 6. Sandboxed Execution & Guardrails
- 7. Churn Prediction — Dataset & Pipeline
- 8. Modeling & Results
- 9. Feature Importance
- 10. Discussion, Limitations, Future Work
- 11. Conclusion & Acknowledgments

1. Data-access bottleneck

- Stakeholders depend on analysts to write SQL for every new question.
- Round-trip slows decisions by hours to days.
- Cognitive cost of SQL gates which questions get asked at all.
- Companies become biased toward questions that fit existing dashboards.

2. Reactive retention

- Churn is discovered AFTER revenue is lost.
- Acquiring a new customer costs 5-25× more than retention.
- Most retention programs only kick in after a bad quarter.
- They target the wrong cohort because they rely on intuition.

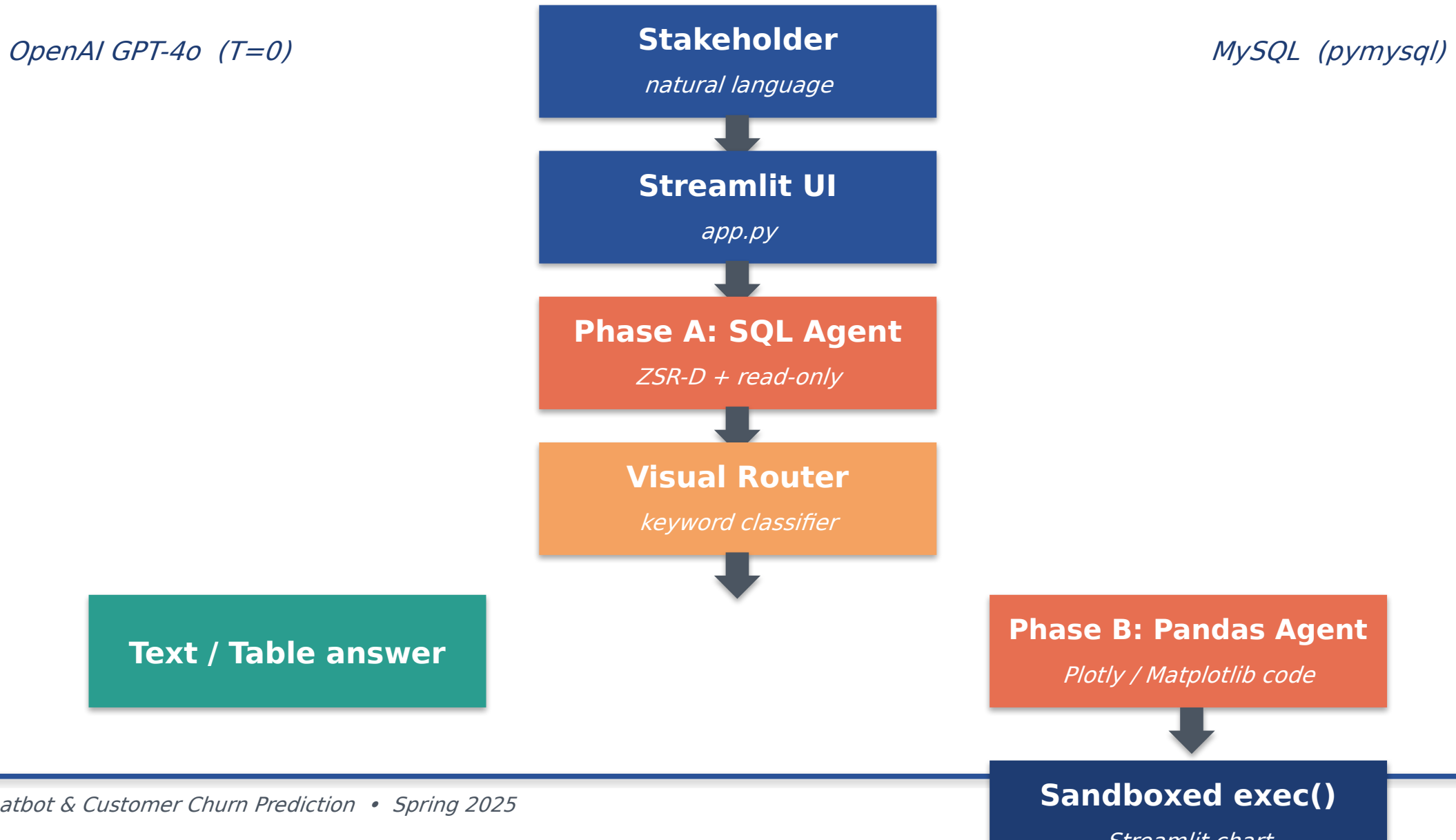
Functional

- Translate plain English into syntactically valid MySQL.
- Dynamic output: table / summary / interactive chart.
- Multi-turn conversational memory across follow-ups.
- Predict churn and rank customers by retention risk.

Non-functional

- Strict read-only safety — no INSERT/UPDATE/DELETE/DROP.
- Reliability — no generated code can crash the UI.
- Performance — cache DB pool and LLM client across reruns.
- Reproducibility — fixed-seed dataset, leak-free CV.
- Interpretability — surface drivers, not just scores.

System Architecture — Dual-Agent Pipeline



Phase A — The SQL Agent

- **Built with `create_sql_agent()`**
- Agent type: `ZERO_SHOT_REACT_DESCRIPTION`.
- Transparent Thought → Action → Observation loop.
- Read-only system prefix forbids `INSERT/UPDATE/DELETE/DROP/...`
- `handle_parsing_errors=True` (recoverable parse failures).
- `max_iterations=12` (prevents runaway billing).
- `sample_rows_in_table_info=3` (column disambiguation).
- `@st.cache_resource` keeps the SQLAlchemy pool warm across reruns.

Thought: `list tables`

Action: `list_tables`

Obs: `customers, orders, ...`

Thought: `inspect schema`

Action: `schema(customers)`

Obs: `columns + sample rows`

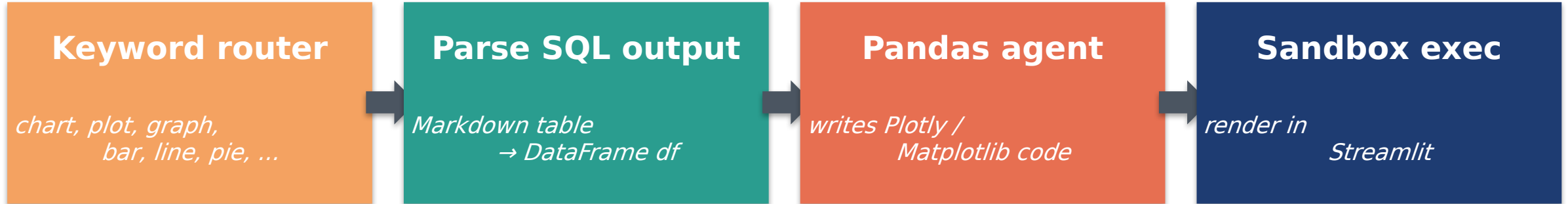
Thought: `draft SELECT`

Action: `query(SELECT ...)`

Obs: `result rows`

Final: `summary + table`

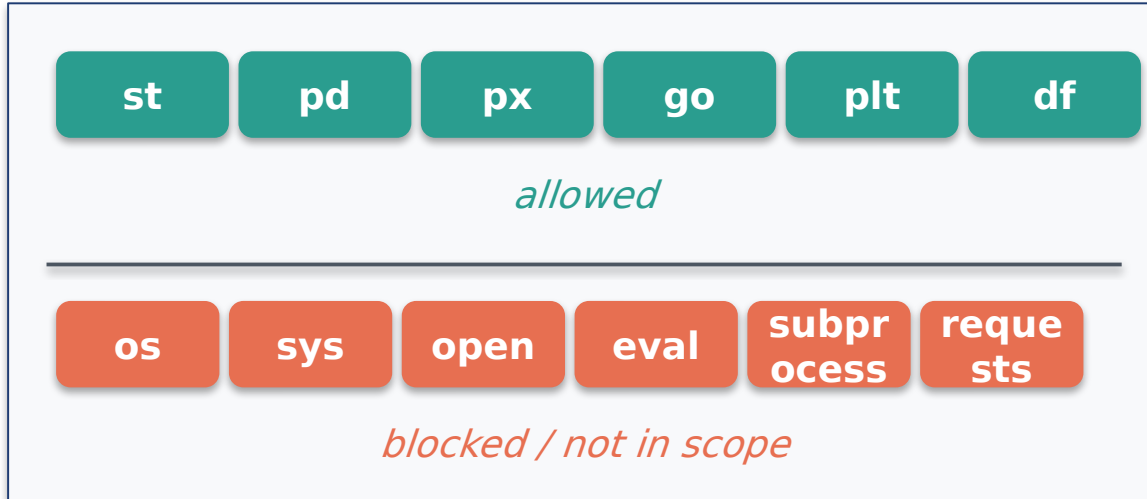
Two-step visualization pipeline:



Why a deterministic keyword router?

- Free — no additional API call per turn.
- Auditable — the same prompt always routes the same way.
- Sufficient — the visual vocabulary in real questions is narrow.
- Trade-off: lower recall on paraphrases; we accept this as an explicit limitation.

exec() sandbox namespace



Three-layer defense

1. Prompt-level

System prompt forbids imports, os, sys, subprocess, open, eval, network APIs.

2. Namespace-level

exec() only sees six names; everything else simply doesn't exist.

3. Exception-level

Broad try/except converts crashes into a friendly Streamlit error card.

Read-only SQL + Conversational memory

- System prefix enumerates and forbids every mutation verb.
- Defense-in-depth: deploy with a read-only MySQL role too.
- st.session_state.messages persists turns across reruns → true follow-ups.

Chatbot — Sample Interaction

USER

Show me the top 10 customers by revenue for 2024.

ASSISTANT

Returned a 10-row table sorted by total revenue (descending). Top customer: \$48,212.

USER

Now break that out by region and plot it as a bar chart.

ASSISTANT

Generated Plotly bar chart — 4 regions x 10 customers, rendered in-line.

USER

Change it to a line chart by month.

Synthetic 5,000-customer e-commerce dataset

seed=42 • fully reproducible • ~27% positive rate

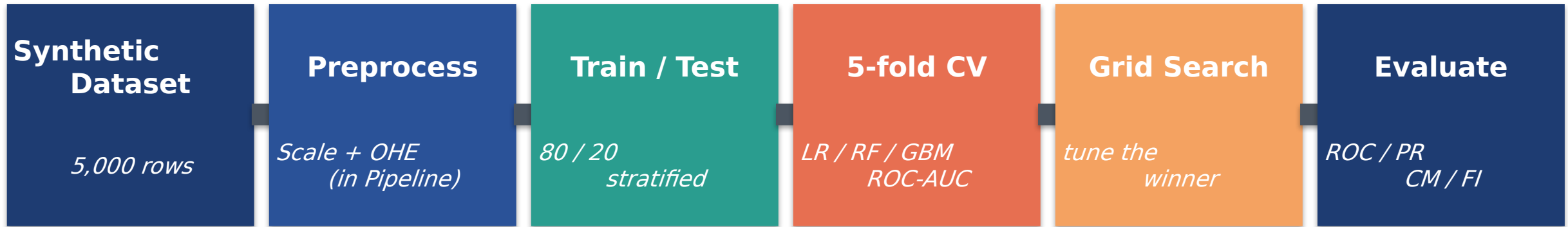
Demographic	age, gender, region
Account	membership tier, payment, tenure_months
Behavior	avg_order_value, orders_per_month
Behavior	return_rate, support_tickets
Behavior	days_since_last_order, discount_usage
Target	churned (0/1)

Churn label

```
s = 0.020 · recency
+ 1.500 · return_rate
+ 0.150 · tickets
- 0.030 · tenure
- 0.400 · 1[Platinum]
- 0.200 · 1[Gold]
- 0.060 · orders/mo
+ N(0, 0.5)
```

```
y = 1{ s > 73rd percentile }
```

Modeling Pipeline (leakage-free)



Why this is leakage-free

- ColumnTransformer (scaler + OHE) lives INSIDE the sklearn Pipeline.
- scikit-learn re-fits the preprocessor on every CV fold separately.
- Scaling parameters never leak from the validation fold into training.
- If we fit the preprocessor once on all training data → optimistic CV scores.

Results — 5-Fold Cross-Validation

Mean CV ROC-AUC

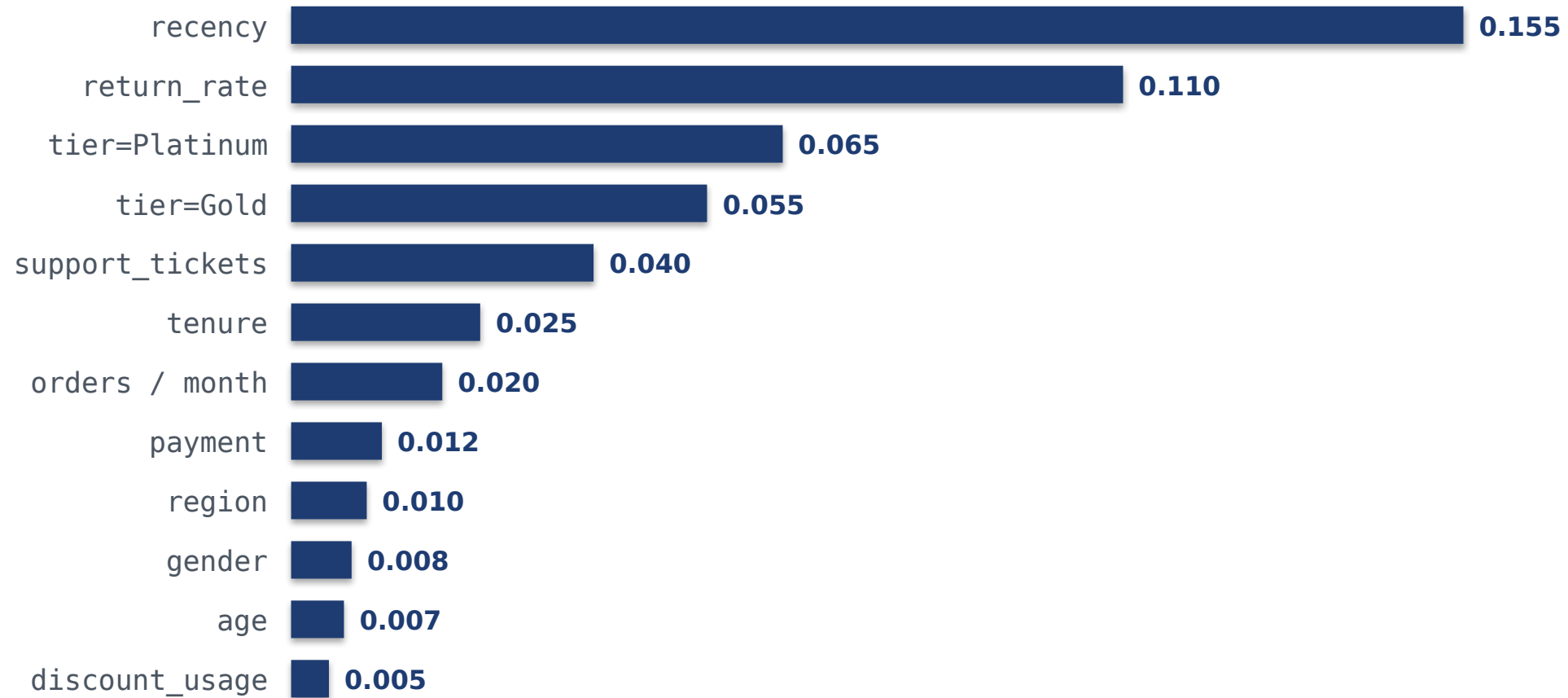


Cross-validation summary

Model	ROC-AUC	Std
Gradient Boosting	0.87	0.010
Random Forest	0.85	0.011
Logistic Reg.	0.80	0.012

Tuned Gradient Boosting wins. Ensembles beat the linear baseline by a clear margin on the held-out test set as well.

Top Churn Drivers — Permutation Importance



Recency and return rate dominate; loyalty tier (Gold, Platinum) protects.

Discussion — Why the Two Halves Reinforce

- Part I lets the retention manager ASK: 'top 200 at-risk customers in Q4, by region.'
- Part II PROVIDES the predictive signal behind that ranking.
- Same data domain → a virtuous loop between exploration and prediction.
- Read-only guardrails + sandboxed exec satisfy enterprise security teams.
- Two architectural decisions paid off the most:
 - → splitting Phase A (SQL) from Phase B (viz) instead of fusing them.
 - → parsing SQL output to a real DataFrame before invoking the visualization agent.
- Modular artifacts (one app.py, one .ipynb) maximize portability and reviewability.

Limitations

- `exec()` is not a true sandbox — trust the LLM provider.
- Synthetic data, not yet validated on a live warehouse.
- Keyword router misses paraphrased visual intent.
- Default 0.5 probability threshold is not cost-tuned.

Future Work

- Replace keyword router with an LLM-as-judge intent classifier.
- Retrain on the live MySQL warehouse via the same connection.
- Add per-customer SHAP attributions for individualized offers.
- Add a 'narrate this chart' step closing the loop with the LLM.
- Container-isolated execution for truly untrusted environments.
- Wire up LangSmith / Helicone observability with token + latency tracking.

Thank You

Questions • Discussion • Feedback

Tuyiramye Christian (2517025) • Dushime Pacifique (2517004)

Supervisor: Prof. Dr. Jebran Khan

Kyungdong University Global • Department of AI • Spring 2025